

Unit 13: Product Metrics

CONTENTS

Objectives

Introduction

13.1 Software Metrics

13.2 Product Metrics

13.3 Product Quality Metrics

13.4 Software Measurement

13.5 Measurement Principles

13.6 ISO Standards

Summary

Keywords

Self Assessment

Answer for Self Assessment

Review Questions

Further Reading

Objectives

After studying this unit, you will be able to:

- Describe measure, metrics and indicators
- Demonstrate measurement principles
- Metrics for specification quality

Introduction

Metrics are only useful if they have undergone statistical validation. The control chart is a simple way to do this while also looking at the variation and distribution of metrics findings. Culture changes as a result of measurement. To launch a metrics programme, three steps must be completed: data collecting, metrics computation, and metrics analysis. A goal-driven strategy, in general, assists an organization in focusing on the correct metrics for its business. Software engineers and their managers can obtain a better understanding of the work they do and the product they make by developing a metrics baseline, which is a database comprising process and product measurements. An indicator is a measure or set of metrics that provides information about the software development process, a software project, or a product. An indicator gives information that allows the project manager or software engineers to improve the process, the project, or the process by adjusting the process, the project, or the process.

13.1 Software Metrics

A software metric is a quantitative or countable measure of programme properties. Software metrics are useful for a variety of purposes, including evaluating software performance, planning work items, calculating productivity, and more. Software metrics are useful for a variety of purposes, including assessing software performance, planning work items, and determining productivity. Many indicators are intertwined throughout the software development process. Software metrics are comparable to management's four functions: planning, organization, control, and improvement.

Characteristics of software Metrics

Quantitative: Metrics must possess quantitative nature. It means metrics can be expressed in values.

Understandable: Metric computation should be easily understood, the method of computing metric should be clearly defined.

Applicability: Metrics should be applicable in the initial phases of development of the software.

Repeatable: The metric values should be same when measured repeatedly and consistent in nature.

Economical: Computation of metric should be economical.

Language Independent: Metrics should not depend on any programming language

13.2 Product Metrics

Product metrics are software product measures at any stage of their development, from requirements to established systems. Product metrics are related to software features only.

Why have Software Product Metrics?

- Help software engineers to better understand the attributes of models and assess the quality of the software
- Help software engineers to gain insight into the design and construction of the software
- Focus on specific attributes of software engineering work products resulting from analysis, design, coding, and testing
- Provide a systematic way to assess quality based on a set of clearly defined rules
- Provide an “on-the-spot” rather than “after-the-fact” insight into the software development

Types of Metrics

Internal metrics: Internal metrics are the metrics used for measuring properties that are viewed to be of greater importance to a software developer. For example, Lines of Code (LOC) measure.

External metrics: External metrics are the metrics used for measuring properties that are viewed to be of greater importance to the user, e.g., portability, reliability, functionality, usability, etc.

Hybrid metrics: Hybrid metrics are the metrics that combine product, process, and resource metrics. For example, cost per FP where FP stands for Function Point Metric.

Project metrics: Project metrics are the metrics used by the project manager to check the project's progress. Data from the past projects are used to collect various metrics, like time and cost; these estimates are used as a base of new software.

Software Metrics classification

Product metrics – Describes the characteristics of the product such as size, complexity, design features, performance, and quality level.

Process metrics – these characteristics can be used to improve the development and maintenance activities of the software.

Project metrics – this metrics describe the project characteristics and execution. Examples include the number of software developers, the staffing pattern over the life cycle of the software, cost, schedule, and productivity.

Product metrics fall into two classes:

Dynamic metrics that are collected by measurements made from a program in execution. Dynamic metrics are usually quite closely related to software quality attributes. It is relatively easy to measure the execution time required for particular tasks and to estimate the time required to start the system. These are directly related to the efficiency of the system failures and the type of failure can be logged and directly related to the reliability of the software.

Static metrics that are collected by measurements made from system representations such as design, programs, or documentation. Static matrices have an indirect relationship with quality attributes. A large number of these matrices have been proposed to try to derive and validate the relationship between the complexity, understandability, and maintainability. Several static metrics which have been used for assessing quality attributes.

Software Product Metrics:

Fan-in/Fan-out

Fan-in is a measure of the number of functions that call some other function (say X). Fan-out is the number of functions which are called by function X. A high value for fan-in means that X is tightly coupled to the rest of the design and changes to X will have extensive knock-on effects. A high value for fan-out suggests that the overall complexity of the control logic needed to coordinate the called components.

Length of code

This is measure of the size of a program. Generally, the large the size of the code of a program component, the more complex and error-prone that component is likely to be.

Cyclomatic complexity

This is a measure of the control complexity of a program. This control complexity may be related to program understandability. It is invented by Thomas McCabe (1974) to measure the complexity of a program's conditional logic. It counts the number of decisions in the program, under the assumption that decisions are difficult for people. It makes assumptions about decision-counting rules and linear dependence of the total count to complexity.

Cyclomatic complexity, also known as V(G) or the graph theoretic number, is probably the most widely used complexity metric in software engineering. Defined by Thomas McCabe, it's easy to understand and calculate, and it gives useful results. This metric considers the control logic in a procedure. It's a measure of structural complexity. Low complexity is desirable.

How to calculate cyclomatic complexity?

CC = Number of decisions + 1

The cyclomatic complexity of a procedure equals the number of decisions plus one. What are decisions? Decisions are caused by conditional statements. In Visual Basic they are If. Else If. Else, Case, For. Next, Until, While, Catch and When. In order to get CC, one simply counts the conditional statements. A multi-way decision, the Select Case block, typical counts as several conditional statements. The decisions counted for each statement or construct are listed below.

Decisions Counted for each Statement

Construct	Decisions	Reasoning
If..Then	+1	An "If statement" is a single decision.
ElseIf..Then	+1	ElseIf adds a new decision.
Else	0	Else does not cause a new decision. The decision is at the If.
#If..#ElseIf..#Else	0	Conditional compilation adds no run-time decisions.
Select Case	0	Select Case initiates the following Case branches, but does not add a decision alone.
Case	+1	Each Case branch adds a new decision.
Case Else	0	Case Else does not cause a new decision. The decisions were made at the other Cases.
For [Each] .. Next	+1	There is a decision at the For statement.
Do While Until	+1	There is a decision at the start of the Do..Loop.
Loop While Until	+1	There is a decision at the end of the Do..Loop.
Do..Loop alone	0	There is no decision in an unconditional Do..Loop without While or Until.*
While	+1	There is a decision at the start of the While..Wend or While..End While loop.
Catch	+1	Each Catch branch adds a new conditional path of execution. Even though a Catch can be either conditional (catches specific exceptions) or unconditional (catches all exceptions), we treat all of them the same way.*
Catch..When	+2	The When condition adds a second decision.*

The minimum limit for cyclomatic complexity is 1. This happens with a procedure having no decisions at all. There is no maximum value since a procedure can have any number of decisions.

Variations: CC, CC2 and CC3

Cyclomatic complexity comes in a couple of variations as to what exactly counts as a decision. Project Analyzer supports three alternative cyclomatic complexity metrics. CC is the basic version. CC2 and CC3 use slightly different rules.

CC does not count Boolean operators such as And and Or. Boolean operators add internal complexity to the decisions, but they are not counted in CC. CC and CC3 are similar what comes to Booleans, but CC2 is different.

CC2 Cyclomatic Complexity with Booleans (“Extended Cyclomatic Complexity”)

$CC2 = CC + \text{Boolean operators}$

CC2 extends cyclomatic complexity by including Boolean operators in the decision count.

Whenever a Boolean operator (And, Or, Xor, Eqv, And Also, Or Else) is found within a conditional statement, CC2 increases by one. The statements considered are: If, Else If, Select, Case, Until, While, When.

The reasoning behind CC2 is that a Boolean operator increases the internal complexity of a decision. CC2 counts the “real” number of decisions, regardless of whether they appear as a single conditional statement or split into several statements. Instead of using Boolean operators to combine decisions into one ($x = 1 \text{ And } y = 2$), you could as well split the decisions into several sub-conditions (If $x = 1$ Then If $y = 2$ Then). CC2 is immune to this kind of restructuring, which might be well justified to make the code more readable. On the other hand, one can decrease CC simply by combining decisions with Boolean operators, which may not make sense. Including Boolean operators in cyclomatic complexity was originally suggested by Thomas McCabe. In this sense, both CC and CC2 are “original” cyclomatic complexity measures. Alternative names: CC2 is also known as ECC extended cyclomatic complexity or strict cyclomatic complexity.

CC3 Cyclomatic Complexity without Cases (“Modified Cyclomatic Complexity”)

$CC3 = CC$ where each Select block counts as one

CC3 equals the regular CC metric, but each Select Case block is counted as one branch, not as multiple branches. In this variation, a Select Case is treated as if it were a single big decision.

This leads to considerably lower complexity values for procedures with large Select Case statements. In many cases, Select Case blocks are simple enough to consider as one decision, which justifies the use of CC3.

Alternative Name: CC3 is sometimes called modified cyclomatic complexity.

Summary of cyclomatic complexity metrics is shown in the table.

Summary of Cyclomatic Complexity Metrics

Metric	Name	Boolean operators	Select Case	Alternative name
CC	Cyclomatic complexity	Not counted	+1 for each Case branch	Regular cyclomatic complexity
CC2	Cyclomatic complexity with Booleans	+1 for each Boolean	+1 for each Case branch	Extended or strict cyclomatic complexity
CC3	Cyclomatic complexity without Cases	Not counted	+1 for an entire Select Case	Modified cyclomatic complexity

CC, CC2 or CC3 – which one to use? This is your decision. Pick up the one that suits your use best. CC and CC2 are “original” metrics and probably more widely used than CC3. The numeric values are, in increasing order: CC3 (lowest), CC (middle) and CC2 (highest). In a sense, CC2 is the most pessimistic metric. All of them are heavily correlated, so you can achieve good results with any of them.

Cyclomatic Complexity and Select Case

The use of multi-branch statements (Select Case) often leads to high cyclomatic complexity values. This is a potential source of confusion. Should a long multi-way selection be split into several procedures?

McCabe originally recommended exempting modules consisting of single multi-way decision statements from the complexity limit.

Although a procedure consisting of a single multi-way decision may require many tests, each test should be easy to construct and execute. Each decision branch can be understood and maintained in isolation, so the procedure is likely to be reliable and maintainable. Therefore, it is reasonable to exempt procedures consisting of a single multi-way decision statement from a complexity limit.

Total Cyclomatic Complexity (TCC)

The total cyclomatic complexity for a project or a class is calculated as follows:

$$\text{TCC} = \text{Sum (CC)} - \text{Count (CC)} + 1$$

TCC equals the number of decisions + 1 in a project or a class. It's similar to CC but for several procedures.

Sum (CC) is simply the total sum of CC of all procedures. Count (CC) equals the number of procedures. It's deducted because we already added +1 in the formula of CC for each procedure.

TCC is immune to modularization, or the lack of modularization. TCC always equals the number of decisions + 1. It is not affected by how many procedures the decisions are distributed.

TCC can be decreased by reducing the complexity of individual procedures. An alternative is to eliminate duplicated or unused procedures.

DECDENS (Decision Density)

Cyclomatic complexity is usually higher in longer procedures. How much decision is there actually, compared to lines of code? This is where you need decision density, which also known as cyclomatic density.

$$\text{DECDENS} = \text{Sum(CC)}/\text{LLOC}$$

This metric shows the average cyclomatic density in your project. The numerator is sum of CC over all your procedures. The denominator is the logical lines of code metric. DECDENS ignores single-line procedure declarations since cyclomatic complexity isn't defined for them.

Length of identifiers

This is a measure of the average length of distinct identifier in a program. The longer the identifiers, the more understandable the program.

Depth of conditional nesting

This is a measure of the depth of nesting of if statements in a program. Deeply nested if statements are hard to understand and are potentially error-prone.

Fog index

This is a measure of the average length of words and sentences in documents. The higher the value for the Fog index, the more difficult the document may be to understand.

13.3 Product Quality Metrics

Mean Time to Failure: It is the time between failures. This metric is mostly used with safety critical systems such as the airline traffic control systems, avionics, and weapons.

Defect Density: It measures the defects relative to the software size expressed as lines of code or function point, etc. i.e., it measures code quality per unit. This metric is used in many commercial software systems.

Customer Problems: It measures the problems that customers encounter when using the product. It contains the customer's perspective towards the problem space of the software, which includes the non-defect oriented problems together with the defect problems.

Customer Satisfaction: Customer satisfaction is often measured by customer survey data through the five-point scale

Very satisfied

Satisfied

Neutral

Dissatisfied

Very dissatisfied

Advantages of software metrics

- The design methodology of the software systems can be studied comparatively.
- The characteristics of various programming languages can be studied for analysis and comparison using software metrics.
- The software quality specifications can be prepared using software metrics.
- The compliance of requirements and specifications of software systems can be verified.
- The complexity of the code can be determined.
- The decision of whether to divide a complex module or not can be done
- The utilization of resource managers to their fullest can be guided.
- Design trade-offs and comparing maintenance costs and software development costs can be done.

Disadvantages of software metrics

- The available tools and the working environment is used to define and derive the software metrics, and there is no standard in defining and deriving them.
- Certain variables are estimated based on the predictive models, and they are not known so often.
- It is not easy to apply metrics in all cases. It is difficult and expensive in some cases.
- It is difficult to verify the validity of historical or empirical data on which the verification and justification.
- Software products can be managed, but the technical staff's performance cannot be evaluated using software metrics.

A measure provides a quantitative indication of the extent, amount, dimension, capacity, or size of some attribute of a product or process

As per IEEE a metric is "a quantitative measure of the degree to which a system, component, or process possesses a given attribute"

An indicator is a metric or combination of metrics that provide insight into the software process, a software project, or the product itself

13.4 Software Measurement

A measurement is an manifestation of the size, quantity, amount or dimension of a particular attributes of a product or process. It is an authority within software engineering. Software measurement process is defined and governed by ISO Standard.

The most important product metric is its size. Size may be expressed in terms of the LOC, number of classes or number of bytes of source code. But LOC is the most popular metric. Software is measured for the following reasons:

- (a) To indicate the quality of the product.
- (b) To assess the productivity of the people.
- (c) To form a baseline for estimation.
- (d) To justify the needs of new tools and staff training.
- (e) To access advantages for new tools and methods.

Size-Oriented Metrics/LOC Metrics/Direct Measures

LOC or thousands of LOC (KLOC) has been a very traditional and direct method of metrics of software. These metrics are derived by normalizing the quality and productivity measures by considering the size of the product as a metric. Please remember the following points regarding

LOC measures:

1. In size-oriented metrics, LOC is considered to be the normalization value.
2. It is an older method that was developed when FORTRAN and COBOL programming were very popular.
3. Productivity is defined as $KLOC / EFFORT$ where effort is measured in person-months.
4. Size-oriented metrics depend on the programming language used.
5. As productivity depends on KLOC, so assembly language code will have more productivity.
6. LOC measure requires a level of detail which may not be practically achievable.
7. The more expressive is the programming language, the lower is the productivity.
8. LOC method of measurement is not applicable to projects that deal with visual (GUI based) programming. As already explained, Graphical User Interfaces (GUIs) use forms basically. LOC metric is not applicable there.
9. It requires that all organizations must use the same method for counting LOC. This is so because some organizations use only executable statements, some use comments and some do not. So, a standard needs to be established.
10. These metrics are not universally accepted.

Based on the LOC/KLOC count of software, many other metrics can be computed:

- (a) Errors/KLOC.
- (b) \$/ KLOC.
- (c) Defects/KLOC.
- (d) Pages of documentation/KLOC.
- (e) Errors/PM.
- (f) Productivity = $KLOC/PM$ (effort is measured in person-months).
- (g) \$/ Page of documentation.

Advantages and disadvantages of LOC measure:

Proponents (in favor) of LOC measure claim that LOC is an artifact of all software development projects that can be easily counted.

Opponents (against), however, argue that LOC measures are programming language dependent. That is, a C program will take lesser LOC as compared to a COBOL/assembly language program for a given task. For example, a bubble sort program in C will take lesser number of LOC as compared to a COBOL program of bubble sort.

Text-Based Metrics

Another metrics that may be used is the size of the text of SRS. The size could be the number of pages, number of paragraphs, number of functional requirements, etc.

Advantages and disadvantages of text-based measure:

Proponents (in favor) of text-based metric say that it is used mostly to convey a general sense about the size of the product or project.

Opponents (against) say that such metrics are dependent on the authors of the document and are not the accurate indicators of the size of the project.

Function Oriented Metrics

In 1977, A. J. Albrecht of IBM developed a method of software metrics based on the functionality of the software delivered by an application as a normalization value. He called it the Function Points (FPs). They are derived using an empirical relationship based on direct measures of software's

information domain and assessments of software complexity. Note that FPs try to quantify the functionality of the system, i.e., what the system performs. This is taken as the method of measurement as FPs cannot be measured directly. Also note that FP is not a single characteristic but is a combination of several software features/characteristics. A non-profit organization called International Function Point User Group (IFPUG) was formed in 1986 that took the responsibility of developing the counting rules and FP definitions. Today, IFPUG has more than 3000 members and has affiliates in 24 countries. FP also has many variants like backfired FPs, COSMIC FPs, Feature Points, The Netherlands FPs, etc. Remember the following points regarding FPs:

1. FPs of an application is found out by counting the number and types of functions used in the applications. Various functions used in an application can be put under five type

Types of FP Attributes

Measurement Parameter	Examples
1. Number of external inputs (EI) 2. Number of external outputs (EO) 3. Number of external inquiries (EQ) 4. Number of internal files (ILF) 5. Number of external interfaces (EIF)	Input screen and tables. Output screens and reports. Prompts and interrupts. Databases and directories. Shared databases and shared routines.

All these parameters are then individually assessed for complexity.

2. FP characterizes the complexity of the software system and hence can be used to depict the project time and the manpower requirement.
3. The effort required to develop the project depends on what the software does.
4. FP is programming language independent.
5. FP method is used for data processing systems, business systems like information systems.
6. The 5 parameters mentioned above are also known as information domain characteristics.
7. All the above-mentioned parameters are assigned some weights that have been experimentally determined

Measurement Parameter	Low	Average	High
1. Number of external inputs (EI)	7	10	15
2. Number of external outputs (EO)	5	7	10
3. Number of external inquiries (EQ)	3	4	6
4. Number of internal files (ILF)	4	5	7
5. Number of external interfaces (EIF)	3	4	6

Weights of 5-FP Attributes

The functional complexities are multiplied with the corresponding weights against each function and the values are added up to determine the UFP (Unadjusted Function Point) of the subsystem.

Computing FPs

Measurement Parameter	Count		Weighing factor			
			Simple Average Complex			
1. Number of external inputs (EI)	—	*	3	4	6 =	—
2. Number of external outputs (EO)	—	*	4	5	7 =	—
3. Number of external inquiries (EQ)	—	*	3	4	6 =	—
4. Number of internal files (ILF)	—	*	7	10	15 =	—
5. Number of external interfaces (EIF)	—	*	5	7	10 =	—
Count-total →						—

Note here that weighing factor will be simple, average or complex for a measurement parameter type.

The Function Point (FP) is thus calculated with the following formula

$$\begin{aligned} \text{FP} &= \text{Count-total} * [0.65 + 0.01 * \sum(\text{Fi})] \\ &= \text{Count-total} * \text{CAF} \end{aligned}$$

Where Count-total is obtained from the above Table.

$$\text{CAF} = [0.65 + 0.01 * \sum(\text{Fi})]$$

and $\sum(\text{Fi})$ is the sum of all 14 questionnaires and show the complexity adjustment value/ factor-CAF (where i ranges from 1 to 14). Usually, a student is provided with the value of $\sum(\text{Fi})$. Also note that $\sum(\text{Fi})$ ranges from 0 to 70, i.e.,

$$0 \leq \sum(\text{Fi}) \leq 70$$

and CAF ranges from 0.65 to 1.35 because

(a) When $\sum(\text{Fi}) = 0$ then $\text{CAF} = 0.65$

(b) When $\sum(\text{Fi}) = 70$ then $\text{CAF} = 0.65 + (0.01 * 70) = 0.65 + 0.7 = 1.35$

Based on the FP measure of software many other metrics can be computed:

- (a) Errors/FP
- (b) \$/FP.
- (c) Defects/FP
- (d) Pages of documentation/FP
- (e) Errors/PM.
- (f) Productivity = FP/PM (effort is measured in person-months).
- (g) \$/Page of Documentation.

8. LOCs of an application can be estimated from FPs. That is, they are interconvertible. This process is known as backfiring. For example, 1 FP is equal to about 100 lines of COBOL code.

9. FP metrics is used mostly for measuring the size of Management Information System (MIS) software.

10. But the function points obtained above are unadjusted function points (UFPs). These

(UFPs) of a subsystem are further adjusted by considering some more General System

Characteristics (GSCs). It is a set of 14 GSCs that need to be considered. The procedure for adjusting UFPs is as follows:

- (a) Degree of Influence (DI) for each of these 14 GSCs is assessed on a scale of 0 to 5.
- (b) If a particular GSC has no influence, then its weight is taken as 0 and if it has a strong influence then its weight is 5.
- (c) The score of all 14 GSCs is totaled to determine Total Degree of Influence (TDI).
- (d) Then Value Adjustment Factor (VAF) is computed from TDI by using the formula:

$$\text{VAF} = (\text{TDI} * 0.01) + 0.65$$

Remember that the value of VAF lies within 0.65 to 1.35 because

(a) When $\text{TDI} = 0$, $\text{VAF} = 0.65$

(b) When $\text{TDI} = 70$, $\text{VAF} = 1.35$

(e) VAF is then multiplied with the UFP to get the final FP count:

$$\text{FP} = \text{VAF} * \text{UFP}$$

Extended Function Point (EFP) Metrics

FP metric has been further extended to compute:

(a) Feature points.

(b) 3D function points.

Feature Points:

According to Jones,

1. Feature point is the superset of function point measure that can be applied to systems and engineering software applications.
2. The feature points are used in those applications in which the algorithmic complexity is high like real-time systems where time constraints are there, embedded systems, etc.
3. Feature points are computed by counting the information domain values and are weighed by only single weight.
4. Feature point includes another measurement parameter – ALGORITHM.
5. The table for the computation of feature point.

Feature Point Calculations

Measurement Parameter	Count		Weighing factor	
1. Number of external inputs (EI)	—	*	4	—
2. Number of external outputs (EO)	—	*	5	—
3. Number of external inquiries (EQ)	—	*	4	—
4. Number of internal files (ILF)	—	*	7	—
5. Number of external interfaces (EIF)	—	*	7	—
6. Algorithms used	—	*	3	—
Count-total →				—

The feature point is thus calculated with the following formula:

$$FP = \text{Count-total} * [0.65 + 0.01 * \sum (Fi)]$$

$$= \text{Count-total} * CAF$$

where count-total is obtained from the above table.

$$CAF = [0.65 + 0.01 * \sum (Fi)]$$

and $\sum (Fi)$ is the sum of all 14 questionnaires and show the complexity adjustment value/ factor CAF (where i ranges from 1 to 14). Usually a student is provided with the value of S (Fi).

6. Function point and feature point both represent systems functionality only.

7. For real-time applications that are very complex, the feature point is between 20 and 35% higher than the count determined using function point above.

3D-Function Points

According to Boeing,

1. Three dimensions may be used to represent 3D function points—data dimension, functional dimension and control dimension.
2. The data dimension is evaluated as FPs are calculated. Herein, counts are made for inputs, outputs, inquiries, external interfaces and files.
3. The functional dimension adds another feature—Transformation, that is, the sequence of steps which transforms input to output.
4. The control dimension that adds another feature—Transition that is defined as the total number of transitions between states. A state represents some externally observable mode.

Computing FP's

Measurement Parameter	Count		Weighing factor			
			Simple	Average	Complex	
1. Number of external inputs (EI)	32	*	3	4	6	= 128
2. Number of external outputs (EO)	60	*	4	5	7	= 300
3. Number of external inquiries (EQ)	24	*	3	4	6	= 96
4. Number of internal files (ILF)	8	*	7	10	15	= 80
5. Number of external interfaces (EIF)	2	*	5	7	10	= 14
Count-total →						618

Now F_i for average case = 3.

So sum of all F_i ($i = 1$ to 14) = $14 * 3 = 42$

$FP = \text{Count-total} * [0.65 + 0.01 * \sum (F_i)]$

= $618 * [0.65 + 0.01 * 42]$

= $618 * [0.65 + 0.42]$

= $618 * 1.07$

= 661.26

and feature point = $(32 * 4 + 60 * 5 + 24 * 4 + 80 + 14) * 1.07 + \{12 * 15 * 1.07\}$

= 853.86

Shortcomings of Function Point Metric

Subjective Evaluations: It needs subjective evaluation with a lot of judgement involved.

Conversion need: Many efforts and models are based on LOC, a function point need to be converted.

Less Researched Data: Less research data is available on function point as compared to LOC.

Low Accuracy: It has low accuracy of evaluating as a subjective judgement is involved.

Time consuming: It is a time consuming method as less research data is available which generate low accuracy and less effective results.

Late performance: It is performed after creation of design specification.

Long learning curve: As the learning curve is quite long it's not easy to gain proficiency.

Classification of Software Measurement

Direct Measurement: In direct measurement the product, process or thing is measured directly using standard scale.

Indirect Measurement: In indirect measurement the quantity or quality to be measured is measured using related parameter i.e. by use of reference.

13.5 Measurement Principles

The objectives of measurement should be established before data collection begins

Each technical metric should be defined in an unambiguous manner

Metrics should be tailored to best accommodate specific products and processes

Metrics should be derived based on a theory that is valid for the domain of application (e.g., metrics for design should draw upon basic design concepts and principles and attempt to provide an indication of the presence of an attribute that is deemed desirable)

Activities of a Measurement Process

Formulation: The derivation (i.e., identification) of software measures and metrics appropriate for the representation of the software that is being considered

Collection: The mechanism used to accumulate data required to derive the formulated metrics

Analysis: The computation of metrics and the application of mathematical tools

Interpretation: The evaluation of metrics in an effort to gain insight into the quality of the representation

Feedback: Recommendations derived from the interpretation of product metrics and passed on to the software development team

13.6 ISO Standards

Mark-II – ISO/IEC 20968:2002 Software engineering - MII Function Point Analysis - Counting Practices Manual.

NESMA – ISO/IEC 24570:2005 Software engineering - NESMA function size measurement method version 2.1 - Definitions and counting guidelines for the application of Function Point Analysis.

COSMIC – ISO/IEC 19761:2011 Software engineering. A functional size measurement method.

FiSMA – ISO/IEC 29881:2008 Information technology - Software and systems engineering - FiSMA 1.1 functional size measurement method.

IFPUG – ISO/IEC 20926:2009 Software and systems engineering - Software measurement - IFPUG functional size measurement method.

CASE TOOLS FOR SOFTWARE METRICS

Many CASE tools (Computer Aided Software Engineering tools) exist for measuring software.

They are either open source or are paid tools. Some of them are listed below:

1. Analyst4j tool is based on the Eclipse platform and available as a stand-alone Rich Client

Application or as an Eclipse IDE plug-in. It features search, metrics, analyzing quality, and report generation for Java programs.

2. CCCC is an open source command-line tool. It analyzes C++ and Java lines and generates reports on various metrics, including Lines of Code and metrics proposed by

Chidamber & Kemerer and Henry & Kafura.

3. Chidamber & Kemerer Java Metrics is an open source command-line tool. It calculates the C&K object-oriented metrics by processing the byte-code of compiled Java.

4. Dependency Finder is an open source. It is a suite of tools for analyzing compiled Java code. Its core is a dependency analysis application that extracts dependency graphs and mines them for useful information. This application comes as a command-line tool, a

Swing-based application and a web application.

5. Eclipse Metrics Plug-in 1.3.6 by Frank Sauer is an open source metrics calculation and dependency analyzer plugin for the Eclipse IDE. It measures various metrics and detects cycles in package and type dependencies.

6. Eclipse Metrics Plug-in 3.4 by Lance Walton is open source. It calculates various metrics during build cycles and warns, via the problems view, of metrics 'range violations'.

7. OOMeter is an experimental software metrics tool developed by Alghamdi. It accepts

Java/C# source code and UML models in XMI and calculates various metrics.

8. Simmel is an Eclipse plug-in. It provides an SQL like querying language for object-oriented code, which allows searching for bugs, measure code metrics, etc.

Summary

- The standard of measure for the estimation of quality, progress and health of the software testing effort is called software metrics.
- Types of Software Metrics can be divided into three groups: product metrics, process metrics, and project metrics.
- Cyclomatic Complexity counts the number of decisions in the program, under the assumption that decisions are difficult for people.
- Cyclomatic complexity, also known as $V(G)$ or the graph theoretic number, is probably the most widely used complexity metric in software engineering.
- Cyclomatic complexity comes in a couple of variations as to what exactly counts as a decision.
- Boolean operators add internal complexity to the decisions, but they are not counted in
- CC. CC and CC3 are similar what comes to Booleans, but CC2 is different.
- Cyclomatic complexity equals the minimum number of test cases you must execute to cover every possible execution path through your procedure.
- The properties which are of great importance to a software developer can be measured using the metrics called internal metrics.
- The properties which are of great importance to a user can be measured using the metrics called external metrics. An example is portability, reliability, usability, etc.

Keywords

Metrics	Product metrics
Process metrics	Project metrics.
Function point	External Inputs (EI)
External Output (EO)	External inquiries (EQ)
Internal files (ILF)	External interfaces (EIF)

Self Assessment

1. What are the characteristics of software Metrics?
 - A. Quantitative
 - B. Applicability
 - C. Repeatable
 - D. Above all
2. Which is not type of Software Metrics?
 - A. Internal metrics
 - B. Product metrics
 - C. External metrics
 - D. Project metrics
3. Product metrics classes are_
 - A. Dynamic
 - B. Static
 - C. Both dynamic and static
 - D. None of above
4. Software Product Metrics include
 - A. Fan-in/Fan-out
 - B. Length of code
 - C. Cyclomatic complexity
 - D. All of above

5. Which is not part of Product Quality Metrics?
 - A. Defect Density
 - B. Product metrics
 - C. Customer Problems
 - D. Customer Satisfaction

6. What are the point scale for Customer Satisfaction?
 - A. Very satisfied
 - B. Satisfied
 - C. Neutral
 - D. Above all

7. Why we need of software measurement?
 - A. Create the quality of the current product or process.
 - B. Anticipate future qualities of the product or process.
 - C. Enhance the quality of a product or process.
 - D. Above all

8. Software Measurement include?
 - A. Direct Measurement
 - B. Indirect Measurement
 - C. Both direct and indirect measurement
 - D. None of above

9. Which is not activities of a measurement process
 - A. Testing
 - B. Formulation
 - C. Collection
 - D. Feedback

10. How many types of data functions
 - A. 4
 - B. 3
 - C. 2
 - D. 5

11. Transaction Functions include?
 - A. External Inputs
 - B. External Outputs
 - C. External Inquiries
 - D. All of above

12. Which is not types of functional point attributes?
 - A. EQ
 - B. AI
 - C. EI
 - D. ILF

13. What are the shortcomings of function point metric?
 - A. Conversion need
 - B. Low Accuracy
 - C. Time consuming
 - D. All of above

14. What are the parameter for computing the unadjusted function point (UFP)
 - A. Number of inputs
 - B. Number of inquiries
 - C. Number of files
 - D. All of above

Answer for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. B | 3. C | 4. D | 5. B |
| 6. D | 7. D | 8. C | 9. A | 10. C |
| 11. D | 12. B | 13. D | 14. D | |

Review Questions

1. What is significance of software metric?
2. Explain the concept of Cyclomatic complexity.
3. Discuss the different variations of Cyclomatic complexity.
4. Discuss Product metrics.
5. Differentiate between Process Metrics Internal Metrics.
6. What are the shortcomings of function point metric?
7. What is functional point attributes?
8. Differentiate between Internal Logical Files and External Interface Files.



Further Reading

- Books Rajib Mall, Fundamentals of Software Engineering, 2nd Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering – A Practitioner’s Approach, 5th Edition, Tata McGraw Hill Higher education.
- Sommerville, Software Engineering, 6th Edition, Pearson Education



Online Links

- <http://c2.com/cgi/wiki?CyclomaticComplexityMetric>
- <http://www.guru99.com/cyclomatic-complexity.html>
- <http://sunnyday.mit.edu/16.355/classnotes-metrics.pdf>
- <https://www.javatpoint.com/software-engineering-software-metrics>
- https://www.tutorialspoint.com/software_quality_management/software_quality_management_metrics.htm